



Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*
Example Logic: IF `food_here` THEN `suck`; IF `wall_ahead` THEN `turn_left`; ELSE `move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF `wall_ahead` AND `left_is_visited` THEN `turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A mathematically perfect Table-Driven Agent is impractical because the number of possible percept sequences grows exponentially as the agent's lifetime increases. In a complex environment such as Chess, there are an enormous number of possible board states and action sequences. Storing a table containing an action for every possible percept sequence would require an unrealistic amount of memory and computation. Therefore, instead of storing every possible situation, we use an Agent Program with rules, models, or algorithms to determine suitable

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

The if-elif-else statements in the `sense_and_act()` method represent the Condition-Action Rules. Each condition checks the current percept, and the corresponding action is executed when that condition is true.

```
if percept['food_here'] → Condition: Food is present → Action: Suck
elif percept['wall_ahead'] → Condition: Wall is ahead → Action: Turn left
else → Condition: No food and no wall → Action: Move forward
```

This is a Simple Reflex Agent because it selects an action using only the current percept, without maintaining any internal history or state.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The Simple Reflex Agent can become trapped because the environment is Partially Observable and the agent does not maintain any percept history or internal state. The agent only receives local information such as whether there is food or a wall ahead. It does not know its exact position, remember cells it has already visited, or remember previous actions. Therefore, when it encounters the same local situation again, it makes the same decision again. For example, if the agent repeatedly reaches a corner and detects a wall ahead, its rule always tells it to turn left. Since it has no memory that it has already experienced this situation, it can repeatedly follow the same sequence of actions and become trapped in a cycle. Therefore, the combination of partial observability + no memory prevents the Simple Reflex Agent from recognizing that it is repeating a previous situation.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

The `ModelBasedAgent` maintains an internal state, such as a set of visited cells, to remember information from previous interactions with the environment. The Sensor Model is represented by the percept received from the environment, such as `wall_ahead` and `food_here`. This information tells the agent what it currently observes. The Transition Model is handled by updating the agent's internal state after each action. For example, the agent can record the cells it has visited and remember its previous action. It can then use this information to determine how its actions have changed its situation. By combining the current percept with its stored internal state, the Model-Based Agent can recognize previously visited areas and choose alternative actions. This allows it to escape loops

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING